



Anatomía del agente

Guía 5 · Taller 02 de caso aplicado de IA en industria

De un bucle plano a un grafo con siete nodos, y por qué esa diferencia importa.

Primero, el bucle plano

Veinte líneas en `agente/plano.py`, que se corren con `make plano`: preguntarle al modelo, ejecutar las herramientas que pida, devolverle los resultados, repetir hasta que deje de pedir herramientas. Eso es ReAct entero —pensar, actuar, observar— y **funciona**: llama las herramientas correctas sin que nadie le diga cuáles, encadena unas con otras, encuentra cosas de verdad y escribe un diagnóstico que cualquiera firmaría.

El problema aparece al correrlo tres veces sobre el mismo mundo.

Las cuatro patologías

QUÉ HACE	POR QUÉ PASA
Concluye con lo primero que encuentra	Nada le dice cuándo tiene suficiente
Ordena por la señal más ruidosa	El MAPE tiene los números más grandes
Dramatiza el impacto de negocio	Ninguna cifra tiene que defenderse
Mañana redescubre lo mismo	No hay memoria entre ejecuciones

Ninguna es del modelo. **Todas son de la forma del bucle.**

Y ninguna se arregla con un prompt más largo: quince reglas se cumplen a medias, cuáles se olvidan cambia en cada ejecución, no hay forma de comprobar que las cumplió, y ninguna instrucción hace que la ejecución de mañana sepa lo que dijo la de hoy.

El grafo

NODO	SU ÚNICO TRABAJO
percepcion	Decide qué mirar y llama herramientas. No concluye nada
diagnostico	Convierte la evidencia en una hipótesis, con alcance y severidad
reflexion	Intenta refutar su propia hipótesis
revision	Reescribe el diagnóstico haciéndose cargo de la crítica
recomendacion	Emite acciones concretas
accion	Ejecuta las que una política de código deja pasar. Sin LLM

El flujo va hacia adelante, pero lo único que hay que retener son **las dos flechas que vuelven hacia atrás**: si a la reflexión le falta evidencia, se regresa a `percepcion`; si la crítica quedó en pie, se pasa por `revision` antes de recomendar. Todo lo demás es cableado; esas dos son lo que convierte al agente en algo que se puede creer.

Son dos nodos y no uno porque cuestionarse sin poder corregirse no sirve de nada: el diagnóstico saldría contradiciendo a sus propias objeciones. Y la decisión de a dónde ir después **no la toma el LLM**: la toman un `if` y un contador de vueltas, en código, sobre un campo que el LLM escribió. El modelo redacta el veredicto; el código decide qué hacer con él. Esa distinción es el reto entero.

Por eso una capa no es un prompt: es **un nodo con un solo trabajo, que corre o no corre y deja rastro**. A un prompt que dice «*no dramatics*» no se le puede preguntar si lo cumplió. A un nodo de reflexión sí: o produjo objeciones o no las produjo, y están escritas.

El freno

Cuando el agente puede actuar, equivocarse deja de costar una alerta y pasa a costar 24 modelos reentrenados con datos malos. La política vive en `agente/accion.py`, es **código Python**, y no hay forma de convencerla. Mira dos cosas: la **naturaleza** del problema —si es una anomalía, el fallo está en los datos y reentrenar ahí enseña ruido— y el **radio de daño** —una categoría son 24 modelos y se revierte; la flota son los 192 a la vez, y si el diagnóstico estaba mal no queda ningún modelo sano contra el cual comparar.

Y viene apagado: sin activarlo explícitamente, evalúa qué haría y no hace nada.

El agente propone; estas reglas disponen. Y nunca frenes contra un campo que el propio agente redacta: si el freno depende de algo que el modelo escribe, el modelo puede aflojarlo sin querer.